OmitBench: Controlled Omissions with Call-Site Truth for Auditing Code-Graph Agents

Building, scoring and extending withheld callers in tool answers: a recipe, a verified Python instance and every local Gemma 4 episode

Arjun Thilak R, AI Researcher, Eros Innovation

Ramkumar MV, AI Researcher, Eros Innovation

Kaggle Writeup, Gemma 4 Developer Agent Paper Track, November 2026

Abstract. Code-graph tools give agents caller lists, and those lists are incomplete: a missing caller means unknown, not absent. OmitBench measures both sides of that gap: what a tool drops, and what an agent does about it. Its unit is a controlled omission: a real tool's answer from which exactly one caller with runtime evidence is withheld, every other byte unchanged, scored against call-site truth. We release (i) a recipe and JSON schema for witnessed positives, omission variants, exposure-aware episode records and executable scoring, stating which parts depend on Python; (ii) a Python instance: 750 primary targets in 30 pinned, permissively licensed libraries, with 2,587 confirmed callers at 3,271 call sites, each site listing its witnesses, plus a separate 236-item extension of callers that no name search finds; (iii) graph and lexical baselines that need no model, reported separately for direct and dynamic callers; and (iv) 104,368 write-once episode records of local agents (Gemma 4 and other model families), with token ids. As an audit, LocAgent's own answer misses a test-exercised caller for 136 of the 750 targets. A standard-library verifier re-checks every hash and re-runs every frozen analysis on a CPU. A companion paper reports what the experiments reveal; this Writeup explains how to build, evaluate and extend controlled omissions.

1. Introduction

Repository agents read structured answers: callers, callees, references. Practical program analyses are unsound by design [R5], static Python call graphs miss callees seen at run time [R6], and runtime traces see only executed paths. Caller lists are therefore usually partial. Which true callers does a given tool drop? And when an agent receives a list that drops one, does it find the caller anyway, stop early, or declare the list complete?

Answering either requires ingredients that, to our knowledge, no public resource combines:

- Witnessed positives: callers confirmed per call site, each site naming its evidence, with unconfirmed candidates kept apart: absence of a witness is not a negative label.
- Controlled omissions: answers in a real tool's format that differ from the complete answer in exactly one caller, so a change in behaviour can be attributed to that omission.
- Exposure-aware records: a log of what the model actually saw. An offered tool is often never called [R4], and a "graph" condition whose answer never reached the prompt does not test the graph.
- Executable scoring that needs no model and no human label.

OmitBench supplies each of them; Sections 2–9 give the recipe, schema, v1 instance, an example, baselines, uses, verification and extension. The dataset package is `absent-edges-expA-v1` (https:

//www.kaggle.com/datasets/rarjunthilak/absent-edges-expa-v1); code and protocols are at https://github.com/arjunthilak05/missing-callers-gemma4.

2. Construction recipe

Each step is a separate command keyed by a frozen registry, and each output records the sha256 of its inputs.

1. **Pin libraries with offline tests.** Admit a library whose licence, read at the pin, is permissive and whose own suite passes offline (no network, scrubbed environment) for at least 95% of tests. Pin the newest release tag before a fixed cutoff.
2. **Collect witnessed positives.** Run a pair of static backends and a runtime tracer over the library's own tests. A caller of target X is *confirmed* (set V) if the trace saw it call X or both static backends report it. Every other reported caller is an *unconfirmed candidate* (set U): released, never omitted, never scored as missed.
3. **Select targets and the omitted caller by frozen hash.** Eligibility rules run before any outcome. The omitted caller C* is the first runtime-witnessed caller in a salted sha256 order.
4. **Render variants with the tool's own code.** `natural` (the tool's answer), `vcomplete` (all of V), `omit_node` (C* and its excerpt removed) and `omit_edge` (one match record removed). Between `vcomplete` and `omit_node` every retained entry is byte-identical, including order, ids and excerpts; a mechanical diff rejects any target where this fails.
5. **Run agents behind an isolation boundary.** The runner reads a dispatch view without truth fields, and each episode logs an exposure receipt, the answer blocks its prompt contains, before the first model call.
6. **Score executably.** A submission is a list of (path, line) entries plus a completeness claim; the primary outcome needs only the item's spans.

Table 1. Which components transfer beyond Python.

Component	Language-neutral part	Python-specific in v1	Needed to port
Registry, offline baseline	admission rules, records	uv locks, pytest, `unshare --net`	package and test tooling
Runtime witness	site record and trace format	`sys.monitoring` tracer loaded as a pytest plugin [R13]	a test-time call tracer
Static witnesses	agreement rule (trace, or both backends)	LocAgent's graph [R1], jedi	a pair of independent indexers, such as language servers
Eligibility, omission rules	hashing, omission rules	dunder, property, decorator exclusions	equivalents
Rendering	byte-identity check	LocAgent's answer code	the audited tool's renderer
Harness	OpenAI-compatible endpoint, exposure receipts	none (read-only bash jail needs Linux)	nothing
Scoring, analysis, verifier	JSON in, JSON out	none	nothing

3. Schema

Table 2. Records (JSON or JSONL, UTF-8). Identities are `relpath::qualname`; module-level and class-body code is never a caller.

Record	Key fields
Item	`item_id`, `family`, `commit`, `target` (identity, path, line span), V and U (callers with `sites`, `runtime_witnessed`, `sources`, `stratum`), `omitted` (C* identity, order key, rule, stratum), `variants` (payload sha256 per variant), `lexical_sites`, `membership`, `eligibility`; no source text
Site	`path`, `lineno`, `end_lineno`, `kind` (call, reference or dynamic), `witnesses` (subset of trace, L, J, ast)
Payload	rendered answer per item and variant, canonical JSON
Dispatch row	what the runner may read: item id, payload hashes, wording; no truth
Episode attempt	`attempt_start.json` with the exposure receipt; `events.jsonl` with prompts, responses, token ids and tool input and output; `result.json` with exit and submission; each written once
Score row	`site_recovered`, `function_localized`, `recall_V_callers`, `precision_lower`, `precision_upper`, `complete`, `certifiably_false_completeness`, exit, sha256 of the terminal `result.json`

Evidence confidence is explicit. A site's `witnesses` name the backends that support it, and a caller is runtime-witnessed if any of its sites has a trace. Each submitted entry falls into one category, in order: invalid, duplicate, V site, U site, lexical name match, or unresolved. Unresolved does not mean false. `precision_lower` counts only V sites as correct, so it is a lower bound; `precision_upper` also counts U sites and name matches, but unresolved entries still count against it, so it is a lenient estimate rather than a strict upper bound. `certifiably_false_completeness` fires only when an agent claims "complete" while a runtime-witnessed caller is missing.

4. The v1 instance

Libraries. 30 primary libraries: alembic, click, cmd2, deepdiff, elementpath, genshi, graphql-core, httpx, icalendar, jinja, lark, liquid, markdown, markdown-it-py, mpmath, packaging, param, petl, pyparsing, python-docx, python-pptx, rdflib, rich, segno, sqlite-utils, traitlets, websockets, XlsxWriter, xmlschema and yapf. Licences, read at the pins: MIT (14), BSD-3-Clause (11), BSD-2-Clause (2), Apache-2.0 (2) and Apache-2.0 OR BSD-2-Clause (1). The pilot used two other libraries.

Truth. Backend L is LocAgent's code graph at commit `4935b557`, with call lines located by an AST scan; backend J is jedi 0.20.1's project-scope references; the tracer ran each suite twice (trace-1 canonical, trace-2 a determinism audit). Kept records: 57,801 L edges, 38,384 J references and 63,716 trace records.

Selection. Of 23,911 package functions, 3,590 are eligible: plain functions or methods with 2–12 confirmed callers, at least one runtime-witnessed, whose answers render untruncated within 12,000 bytes and reveal no omitted call site. Exclusions count at the first failing rule (largest: too few confirmed callers, 9,033; dunder methods, 4,811; byte-identity failures: 0). A frozen hash picks 25 targets per library.

Primary set. 750 targets with 2,587 confirmed callers (2,409 runtime-witnessed) at 3,271 sites; 2,909 sites carry a trace witness and 2,251 are supported by the trace, L and J together. Unconfirmed candidates number 3,585 in 316 items, mostly lexical name matches; they are released without a label. C* is a direct call in 708 items and a dynamic call in 40; alias and callback account for 1 each.

Extensions, kept separate. The *hard panel* reruns the builder with `--omit-rule first_not_grep_findable`, which picks as C* the first runtime-witnessed caller none of whose call sites contains X's name; truth, eligibility, rendering and scoring are unchanged. It has 236 items in 27 libraries and 134 distinct omitted callers, all dynamic calls. It shares 58 targets with the primary set, 40 of them with the same C*; deduplicated, primary set and panel hold 946 distinct items. The pilot's 34 items are never pooled. The primary block runs every item with no list and with the incomplete list, plain or with a caution, a not-exhaustive note or lexical evidence. Secondary blocks reuse primary items: complete answers (a positive control), LocAgent's own answers where it misses a tested caller (130 of the 136 such items, those whose answer renders untruncated) and an arm offering a `graph_query` tool (150 items).

Table 3. Episode records (completed cells; none lost to infrastructure). Gemma 4 31B BF16 unless stated; the full design has 13,740 cells.

Run	Scope	Episodes
Pilot	pilot libraries	636
Main, preregistered	full design	13,740
Gemma 4 12B BF16	full design	13,740
Main-track checkpoint `gemma-4-31b-it-qat-w4a16-ct`	full design	13,740
Doubled budget	primary block, one seed	3,750
Hard panel	primary block and complete answers	4,012
Last-response line: none, neutral, deadline	primary block, one seed	11,250
Evidence ledger, off and on	complete and incomplete lists	9,000
Last-line components: neutral, information, instruction, both	no list and incomplete list, one seed	6,000
Review at first submission: neutral, ledger	complete and incomplete lists, one seed	3,000
Doubled budget with deadline line	complete and incomplete lists	4,500
Impact-analysis task	no, incomplete and complete lists, one seed	2,250
Other models (A8): Qwen3-Coder, GLM-4.5-Air, `Nemotron-3-Nano`, Gemma 4 E4B and E2B	no, incomplete and complete lists; no and incomplete lists with the deadline line; one seed	18,750

Together: 104,368 analysed episodes, plus go/no-go gate runs and records of aborted launches.

5. One item, end to end

Item `394daab0827ab2a087b5` (traitlets) has target X = `traitlets/traitlets.py::Int.subclass_init`. V holds the `subclass_init` methods of `Container`, `Dict`, `Tuple` and `Union` and `MetaHasDescriptors.setup_class`; every site is witnessed by the trace, LocAgent and the AST scan, none by jedi. U holds one lexical-only candidate, `HasDescriptors.setup_instance`, released unlabelled. The hash picks C* = `Container.subclass_init`, a direct call. `omit_node` deletes that caller and its excerpt; every other entry stays byte-identical to `vcomplete`.

Main-run score rows at seed 11:

- **No list:** the agent printed a C* call site in its first response, kept searching, and used every response without submitting (NoAnswer).
- **Incomplete list:** it submitted, claimed "complete", and every entry was a confirmed site, but C* was absent: perfect precision, a missed caller, and a completeness claim scored as certifiably false.

- **Complete list:** it submitted every confirmed caller, C* included.

One item shows both finding without submitting and submitting without finding; receipts confirm which answer each prompt contained.

6. Baselines

Reference policies need no model: the answer alone (a graph baseline), the capped lexical block shown in the lexical-evidence condition (at most 20 hits and 2,000 bytes), and a grep for X's names with frozen aliases. Only 40% of lexical-block hits are confirmed call sites. Table 4 adds the Gemma 4 results.

Table 4. Recovery of C*. Policy rows count items; Gemma rows are recovery rates (library-weighted over all primary seeds; the complete-list row uses seed 11 only). "Direct" and "dynamic" split the primary set by C*'s stratum. The first policy row counts items whose answer lists C* itself; the 136 targets of §7 miss any test-exercised caller.

Policy or agent	Primary	Direct	Dynamic	Hard panel
Items	750	708	40	236
LocAgent's own answer lists C*	645	644	0	0
Capped lexical block contains C*	624	623	0	0
Grep with aliases finds C*	710	708	0	0
Gemma 4 31B, no list	56.8%	60%	9%	7.3%
Gemma 4 31B, incomplete list	72.4%	76%	12%	14.2%
Gemma 4 31B, complete list	84.4%	not split	not split	38.3%

On the same primary items, the incomplete list gave 15.5% recovery with the weaker Gemma 4 12B (no list: 0.8%) and 66.6% with the main-track checkpoint. Direct callers are easy for grep yet often missed by agents; dynamic callers defeat every lexical baseline. On the hard panel, where LocAgent, the lexical block and grep all find nothing, an agent must reason beyond names; report it separately, never pooled.

7. Using the resource

Check it (Python 3.11 or later, standard library, CPU): `python -I -B reproduce/verify.py --root . --jobs N` verifies the manifest, every witness and shard member, the chain from score rows to events, and re-runs each frozen analysis byte for byte. The notebook `notebooks/reproduce_results.ipynb` recomputes the central estimates from the per-cell tables.

Audit a retrieval tool. Render the tool's caller answer for each target and score it like a submission: recall of V or of runtime-witnessed callers, and lower and upper precision. On v1, LocAgent's answer misses a runtime-witnessed caller for 136 of 750 targets; `vcomplete` and `omit_node` are positive and negative controls.

Evaluate an agent. The runner (`scripts/expA_run.py`) reads a dispatch view and checkouts rebuilt from the pinned commits; `--dry-run` renders every prompt without a model. Episodes call `/v1/chat/completions` on each `--endpoint`, so any OpenAI-compatible server works; a new model may need a `--model-profile` for its chat template and tool-call format. Then `scripts/expA_score.py score --run-dir RUN --items items.jsonl --repos-root CHECKOUTS --out scores.jsonl` and `scripts/expA_analyze.py analyze` apply the unchanged scoring and analysis. For an agent built elsewhere, `absent_edges.probe.score.score_submission(sites, complete, item, index)`

scores one submission.

Separate search from submission. Episodes keep every tool output and exit, so users can ask when C* evidence first appeared and whether it was then submitted. In the main run, agents searched X's name in at least 99.3% of episodes; among incomplete-list episodes that ran out of responses, at least 44% had already displayed a C* call site. Every main-run submission claimed completeness (9,968 of 9,968 across all blocks; 8,101 of 8,101 in the primary block). A harness line before the last response raised incomplete-list submission to 98.1%: the records separate finishing from finding.

8. Verification and provenance

- **Freeze.** The plan `protocol/EXPA_PREREG.md` (`protocols/` in the code repository; sha256 57991a7e) and its hash lock were committed as 668a90456 at 2026-11-01 04:02:27 UTC, before any primary-library episode. The lock pins items (sha256 f3451a3c), dispatch views, the model-free findability disclosure, the analysis plan and the code release. Each follow-up has its own protocol, committed before its episodes.
- **Releases.** Static truth was built by code release `2adb0b458`, traces by `dd842ab6b`, items by `0b8ee0b7c`; each run's `run.json` binds items, payloads, checkout manifest, prompts, tools and code hashes.
- **Chain.** Each score row names its terminal `result.json` sha256; each result names its `events.jsonl` and `attempt_start.json`; each shard's `INDEX.jsonl` lists every member's sha256; `PROVENANCE.json` gives every file's source and witnesses. Infrastructure attempts are kept.
- **Checks.** The latest verifier run passed 96 of 96 checks; `RELEASE_ID`, the sha256 of `MANIFEST.sha256`, is dd29f6bbde409311cf354b2797e3ec435800f544381f7cf6e1f15eb175bce571. Independent re-analysis code agreed on 921 of 921 values (largest difference 8.9e-15).
- **Licences.** Project code and data are Apache-2.0; upstream excerpts keep their licences (`THIRD_PARTY_NOTICES.md`); Gemma 4 itself is Apache-2.0 per its model card. Weights and upstream trees are excluded; checkouts are rebuilt from the pins and a per-file manifest.
- **Sanitization.** Files are published byte for byte, except that another person's account name was replaced (old and new sha256 in `SANITIZATION.md`). No human annotation was involved.

9. Extending the resource

- A new library enters through one registry record and a re-run of the static, trace and item commands.
- A new omission rule is a builder option, as the hard panel shows.
- A new tool needs only its renderer; the byte-identity check and scorer are unchanged.
- A new task framing, such as impact analysis, changes the prompt template while reusing items and scoring.
- A new language follows Table 1: a test-time tracer, a pair of indexers and the tool's renderer; everything downstream is language-neutral.
- Versions: `v1` never changes; the author issues additions and errata as new versions with a new `RELEASE_ID`.

10. Related work

Code-graph tools for agents are established [R1, R2, R3, R16], and their uneven use documented: in CodeCompass, 58% of trials with graph access made no graph call [R4], and ablations of graph, grep and embedding tools for small agents, Gemma 4 among them, attribute much of the gain to submission behaviour [R15]. Xu measures agents listing every call site with and without a language server [R17].

Call-graph truth exists: DyPyBench provides executable Python projects with static and dynamic call graphs [R6], TraceEval execution-verified code-reasoning tasks [R11], and LLM call-graph inference has been evaluated directly [R18, R20, R21]. These measure program analysis or model reasoning, not agents reading an incomplete tool answer. Language models struggle to notice omissions [R7, R8]; agents adopt corrupted tool returns [R9] but can catch silent tool errors when warned [R10]; omission probes exist for documents [R7] and for safety facts in tool responses [R19]; repo-graph returns structured explanations of absence [R12]. OmitBench differs in combining per-site witnesses with an unconfirmed set, byte-controlled omissions in a production tool's format, a stratum of search-invisible callers, exposure receipts and a complete episode corpus under one executable scorer.

11. Limitations and ethics

- **Truth is positive evidence.** Witnesses cover tested paths and the static backends; V is not a complete call graph, and U is unlabelled.
- **Scope.** Python, pure-Python packages chosen for offline testability, English prompts, one harness, local open models only (most of them Gemma 4; 12 of 17 other models tried could not drive the harness). Episodes are not byte-reproducible under batched serving; scoring and analysis are.
- **Difficulty.** Most primary C* are direct calls that grep finds, so the primary set mostly tests whether an agent looks beyond the answer; the hard panel isolates callers that names cannot find.
- **Content.** Public source at public tags; no hidden tests, gold patches or competition data. rich and httpx also underlie Gemma 4 agent competition tasks.

Disclosure

Only locally served open models were evaluated as agents.

References

- Companion paper. Arjun Thilak R and Ramkumar MV. Missing Callers Under a Response Budget: Finishing Versus Finding in a Local Gemma 4 Agent. Kaggle Writeup, Google – The Gemma 4 Developer Agent Paper Track, 2026.
- R1. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. ACL 2025. doi:10.18653/v1/2025.acl-long.426. Code: https://github.com/gersteinlab/LocAgent at 4935b557326c154bad8e8dcf3747cc8d32d1f387.
- R2. Ouyang et al. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. ICLR 2025. arXiv:2410.14684.
- R3. Lin, Zhou, Yang, Li. How Much Static Structure Do Code Agents Need? A Study of Deterministic Anchoring. arXiv:2606.26979v2, 2026.
- R4. Paipuru. CodeCompass: Navigating the Navigation Paradox in Agentic Code Intelligence. arXiv:2602.20048, 2026.
- R5. Livshits et al. In Defense of Soundiness: A Manifesto. CACM 58(2), 2015. doi:10.1145/2644805.
- R6. Bouzenia, Krishan, Pradel. DyPyBench: A Benchmark of Executable Python Software. Proc. ACM Softw. Eng. 1(FSE), 2024. doi:10.1145/3643742.
- R7. Fu et al. Absence Bench: Language Models Can't See What's Missing. NeurIPS 2025 Datasets and Benchmarks. doi:10.52202/085713-1277.
- R8. Min et al. When Absence Is Evidence: Evaluating Completeness-Sensitive Negative Reasoning in Large Language Models. arXiv:2608.04591, 2026.
- R9. Yang et al. Agents' Overreliance on Unreliable Tools. arXiv:2609.05587v2, 2026.
- R10. Sun et al. Tools Fail: Detecting Silent Errors in Faulty Tools. EMNLP 2024. doi:10.18653/v1/2024.emnlp-

main.790.

- R11. Li et al. An Execution-Verified Multi-Language Benchmark for Code Semantic Reasoning (TraceEval). arXiv:2605.11006, 2026.
- R12. Chahwan. repo-graph, bench/absence. https://github.com/James-Chahwan/repo-graph/tree/0f8ecfa1cc103af030ef2438ca50e9634a30290e/bench/absence.
- R13. Shannon. PEP 669: Low Impact Monitoring for CPython. https://peps.python.org/pep-0669/.
- R14. Google. gemma-4-31B-it model card. https://huggingface.co/google/gemma-4-31B-it.
- R15. Bellam. How Do Small Coding Agents Use Code Graphs and Embeddings? arXiv:2610.33958, 2026.
- R16. Yu et al. CodeNib: A Multi-View Data System for Serving Repository Context to Coding Agents. arXiv:2607.25431, 2026.
- R17. Xu. Does a Language Server Save Tokens for Coding Agents? A Measurement Methodology and Preliminary Study. arXiv:2608.13568, 2026.
- R18. Venkatesh et al. An Empirical Study of Large Language Models for Type and Call Graph Analysis in Python and JavaScript. Empirical Software Engineering 30(6):167, 2025. doi:10.1007/s10664-025-10704-3.
- R19. Zhang et al. EpiSEM-Bench: Evaluating Safety and Evidence Recovery in LLM Agents under Missing Evidence. arXiv:2610.31650, 2026.
- R20. Yuan, Zhang, Chen, Wang. CG-Bench: Can Language Models Assist Call Graph Construction in the Real World? LMPL 2025 (ICFP/SPLASH 2025). https://conf.researchr.org/details/icfp-splash-2025/lmpl-2025-papers/3/CG-Bench-Can-Language-Models-Assist-Call-Graph-Construction-in-the-Real-World-
- R21. Rossi. GraphCode-Bench-500 v0. Hugging Face dataset, Apache-2.0. https://huggingface.co/datasets/Vittori
Bench-500-v0